Name	Vedant Dinkar Patil
UID no.	2024300170
Experiment No.	6

AIM:	Implement Unsupervised Learning Algorithm [Kohonen Self Organizing Feature Maps] & [LVQ Network]
PROBLEM STATEMENT :	For a given Kohonen self-organizing feature map with weights shown in Figure 4: (a) Use the square of the Euclidean distance to find the cluster unit Y_j closest to the input vector $(0.2,0.4)$. Using a learning rate of 0.2, find the new weights for unit Y_j. (b) For the input vector $(0.6,0.6)$ with learning rate 0.1, find the winning cluster unit and its new weights.
Theory:	Kohonen Self-Organizing Feature Maps (SOM) and Learning Vector Quantization (LVQ) networks are core neural network architectures implemented for clustering and pattern recognition tasks. Specifically, a Kohonen SOM uses an unsupervised competitive learning approach to map high-dimensional input vectors onto a lower-dimensional grid by finding the closest cluster unit (Best Matching Unit) via distance metrics and updating its weights using a learning rate, whereas an LVQ network serves as its supervised counterpart where each output unit is assigned a predefined class label, refining the weight vectors by moving them closer to inputs of the correct class and pushing them away when misclassifications occur.

PROGRAM:

Kohenen code :

```
#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;
void printMatrix(const vector<vector<double>>& W) {
    for (int i = 0; i < W.size(); i++) {
        for (int j = 0; j < W[i].size(); j++) {
            cout << fixed << setprecision(2) << W[i][j] << "\t";
        }
        cout << "\n";
    }
}

int main() {
    vector<vector<double>> original_W = {
        {0.3, 0.2, 0.1, 0.8, 0.4},
        {0.5, 0.6, 0.7, 0.9, 0.2}
    };

    int num_iterations;
    cout << "How many input vectors would you like to test? ";
    cin >> num_iterations;
    cout << "\n";

    for (int k = 0; k < num_iterations; k++) {
        vector<vector<double>> W = original_W;
        double x1, x2, alpha;

        cout << "-------------------------------------------------\n";
        cout << "Test Case " << k + 1 << "\n";
        cout << "-------------------------------------------------\n";
        cout << "Enter value for x1: ";
        cin >> x1;
        cout << "Enter value for x2: ";
        cin >> x2;
        cout << "Enter learning rate (alpha): ";
        cin >> alpha;
        cout << "\n";
```

```
    cout << "Original Weights Matrix (W):\n";
    printMatrix(W);
    cout << "\n";

    cout << "Input Vector: x1 = " << x1 << ", x2 = " << x2 << "\n";
    cout << "Learning Rate (alpha) = " << alpha << "\n\n";

    double minDist = 1e9;
    int winnerIndex = -1;

    cout << "Calculated Distances (D(j) = (w1j - x1)^2 + (w2j - x2)^2):\n";
    for (int j = 0; j < 5; j++) {
        double diff1 = W[0][j] - x1;
        double diff2 = W[1][j] - x2;
        double dist = (diff1 * diff1) + (diff2 * diff2);

        cout << "D(" << j + 1 << ") = (" << W[0][j] << " - " << x1 << ")^2 + ("
             << W[1][j] << " - " << x2 << ")^2 = " << dist << "\n";

        if (dist < minDist) {
            minDist = dist;
            winnerIndex = j;
        }
    }

    cout << "\nWeight with smallest distance is D(" << winnerIndex + 1 << ") =
" << minDist << "\n";
    cout << "Winning cluster unit (J) = " << winnerIndex + 1 << "\n\n";
    cout << "Updating weights for Unit " << winnerIndex + 1 << " using
formula: w_j(new) = w_j(old) + alpha * [x - w_j(old)]\n";

    double old_w1 = W[0][winnerIndex];
    double old_w2 = W[1][winnerIndex];

    W[0][winnerIndex] = old_w1 + alpha * (x1 - old_w1);
    W[1][winnerIndex] = old_w2 + alpha * (x2 - old_w2);

    cout << "w1" << winnerIndex + 1 << "(new) = " << old_w1 << " + " <<
alpha << " * (" << x1 << " - " << old_w1 << ") = " << W[0][winnerIndex] <<
```

```
"\n";
        cout << "w2" << winnerIndex + 1 << "(new) = " << old_w2 << " + " <<
alpha << " * (" << x2 << " - " << old_w2 << ") = " << W[1][winnerIndex] <<
"\n\n";

        cout <<
"============================================================\n";
        cout << "FINAL UPDATED WEIGHT MATRIX FOR TEST CASE " << k
+ 1 << ":\n";
        cout <<
"============================================================\n";
        printMatrix(W);
        cout << "\n\n";
    }

    return 0;
}
```

LVQ code:

```
#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;

// Helper function to print vectors cleanly
void printVector(const vector<double>& vec) {
    cout << "[ ";
    for (double val : vec) {
        cout << fixed << setprecision(2) << val << " ";
    }
    cout << "]";
}

int main() {
    vector<vector<double>> W = {
        {0.0, 0.0, 1.0, 1.0},
        {1.0, 0.0, 0.0, 0.0}
    };
    vector<int> weight_classes = {1, 2};
    vector<vector<double>> train_vectors = {
```

```
        {0.0, 0.0, 0.0, 1.0}, // T = 2
        {1.0, 1.0, 0.0, 0.0}, // T = 1
        {0.0, 1.0, 1.0, 0.0}  // T = 1
    };
    vector<int> train_targets = {2, 1, 1};

    double alpha = 0.1;

    cout <<
"=========================================================\n";
    cout << "       LVQ (Learning Vector Quantization)        \n";
    cout <<
"=========================================================\n";
    cout << "Initial Reference Weights:\n";
    cout << "w1 (Class 1): "; printVector(W[0]); cout << "\n";
    cout << "w2 (Class 2): "; printVector(W[1]); cout << "\n\n";

    bool perfectly_trained = false;
    int epoch = 1;
    int max_epochs = 100;

    while (!perfectly_trained && epoch <= max_epochs) {
        cout <<
"=========================================================\n";
        cout << "                  EPOCH " << epoch << "\n";
        cout <<
"=========================================================\n";

        int matches_this_epoch = 0;

        for (size_t i = 0; i < train_vectors.size(); i++) {
            cout << "-----------------------------------------------\n";
            cout << "Training Vector " << i + 1 << ": ";
            printVector(train_vectors[i]);
            cout << " | Target Class (T): " << train_targets[i] << "\n";
            cout << "-----------------------------------------------\n";

            int winnerIndex = -1;
            double minDist = 1e9;
```

```
        for (int j = 0; j < 2; j++) {
            double dist = 0.0;
            for (int d = 0; d < 4; d++) {
                double diff = W[j][d] - train_vectors[i][d];
                dist += diff * diff;
            }
            cout << "D(" << j + 1 << ") to w" << j + 1 << " = " << dist << "\n";

            if (dist < minDist) {
                minDist = dist;
                winnerIndex = j;
            }
        }

        int predictedClass = weight_classes[winnerIndex];
        cout << "\nWinning Unit: J = " << winnerIndex + 1
             << " (Class " << predictedClass << ") with distance = " << minDist <<
"\n";

        if (predictedClass == train_targets[i]) {
            cout << "Match Found! J = T (" << predictedClass << " == " <<
train_targets[i] << ")\n";
            cout << "Updating w" << winnerIndex + 1 << " using addition
(+)...\n";

            for (int d = 0; d < 4; d++) {
                W[winnerIndex][d] += alpha * (train_vectors[i][d] -
W[winnerIndex][d]);
            }
            matches_this_epoch++;
        } else {
            cout << "Mismatch! J != T (" << predictedClass << " != " <<
train_targets[i] << ")\n";
            cout << "Updating w" << winnerIndex + 1 << " using subtraction (-
)...\n";

            for (int d = 0; d < 4; d++) {
                W[winnerIndex][d] -= alpha * (train_vectors[i][d] -
W[winnerIndex][d]);
            }
```

```
            }

            cout << "Updated w" << winnerIndex + 1 << ": ";
            printVector(W[winnerIndex]);
            cout << "\n\n";
        }

        if (matches_this_epoch == train_vectors.size()) {
            perfectly_trained = true;
            cout << ">>> All training vectors correctly classified! Stopping training
process. <<<\n\n";
        }
        epoch++;
    }

    if (!perfectly_trained) {
        cout << ">>> Max epochs reached. Training stopped. <<<\n\n";
    }

    cout <<
"=================================================\n";
    cout << "TRAINING COMPLETED!\n";
    cout << "Final Updated Weights Matrix:\n";
    cout << "w1 (Class 1): "; printVector(W[0]); cout << "\n";
    cout << "w2 (Class 2): "; printVector(W[1]); cout << "\n";
    cout <<
"=================================================\n\n";

    char choice;
    cout << "Do you want to predict output for a custom test vector? (y/n): ";
    cin >> choice;

    while (choice == 'y' || choice == 'Y') {
        vector<double> testVector(4);
        cout << "\nEnter 4 elements for test vector (x1 x2 x3 x4): ";
        for (int d = 0; d < 4; d++) {
            cin >> testVector[d];
        }

        int winnerIndex = -1;
```

```
        double minDist = 1e9;

        cout << "\nCalculated Distances:\n";
        for (int j = 0; j < 2; j++) {
            double dist = 0.0;
            for (int d = 0; d < 4; d++) {
                double diff = W[j][d] - testVector[d];
                dist += diff * diff;
            }
            cout << "D(" << j + 1 << ") to w" << j + 1 << " = " << dist << "\n";

            if (dist < minDist) {
                minDist = dist;
                winnerIndex = j;
            }
        }

        cout <<
"\n=======================================================\n";
        cout << " PREDICTED OUTPUT CLASS: CLASS " <<
weight_classes[winnerIndex] << "\n";
        cout <<
"=======================================================\n\n";

        cout << "Predict another vector? (y/n): ";
        cin >> choice;
    }

    return 0;
}
```

Output:

```
students@students-HP-280-G3-SFF-Business-PC:~/Desktop/2024300170_Vedant$ g++ kohonen.cpp -o kohonen
^[[Astudents@students-HP-280-G3-SFF-Business-PC:~/Desktop/2024300170_Vedant$ ./kohonen
How many input vectors would you like to test? 2

------------------------------------------------
Test Case 1
------------------------------------------------
Enter value for x1: 0.2
Enter value for x2: 0.4
Enter learning rate (alpha): 0.2

Original Weights Matrix (W):
0.30    0.20    0.10    0.80    0.40
0.50    0.60    0.70    0.90    0.20

Input Vector: x1 = 0.20, x2 = 0.40
Learning Rate (alpha) = 0.20

Calculated Distances (D(j) = (w1j - x1)^2 + (w2j - x2)^2):
D(1) = (0.30 - 0.20)^2 + (0.50 - 0.40)^2 = 0.02
D(2) = (0.20 - 0.20)^2 + (0.60 - 0.40)^2 = 0.04
D(3) = (0.10 - 0.20)^2 + (0.70 - 0.40)^2 = 0.10
D(4) = (0.80 - 0.20)^2 + (0.90 - 0.40)^2 = 0.61
D(5) = (0.40 - 0.20)^2 + (0.20 - 0.40)^2 = 0.08

Weight with smallest distance is D(1) = 0.02
Winning cluster unit (J) = 1

Updating weights for Unit 1 using formula: w_j(new) = w_j(old) + alpha * [x - w_j(old)]
w11(new) = 0.30 + 0.20 * (0.20 - 0.30) = 0.28
w21(new) = 0.50 + 0.20 * (0.40 - 0.50) = 0.48

================================================
FINAL UPDATED WEIGHT MATRIX FOR TEST CASE 1:
================================================
0.28    0.20    0.10    0.80    0.40
0.48    0.60    0.70    0.90    0.20
```

```
------------------------------------------------
Test Case 2
------------------------------------------------
Enter value for x1: 0.6
Enter value for x2: 0.6
Enter learning rate (alpha): 0.1

Original Weights Matrix (W):
0.30    0.20    0.10    0.80    0.40
0.50    0.60    0.70    0.90    0.20

Input Vector: x1 = 0.60, x2 = 0.60
Learning Rate (alpha) = 0.10

Calculated Distances (D(j) = (w1j - x1)^2 + (w2j - x2)^2):
D(1) = (0.30 - 0.60)^2 + (0.50 - 0.60)^2 = 0.10
D(2) = (0.20 - 0.60)^2 + (0.60 - 0.60)^2 = 0.16
D(3) = (0.10 - 0.60)^2 + (0.70 - 0.60)^2 = 0.26
D(4) = (0.80 - 0.60)^2 + (0.90 - 0.60)^2 = 0.13
D(5) = (0.40 - 0.60)^2 + (0.20 - 0.60)^2 = 0.20

Weight with smallest distance is D(1) = 0.10
Winning cluster unit (J) = 1

Updating weights for Unit 1 using formula: w_j(new) = w_j(old) + alpha * [x - w_j(old)]
w11(new) = 0.30 + 0.10 * (0.60 - 0.30) = 0.33
w21(new) = 0.50 + 0.10 * (0.60 - 0.50) = 0.51

================================================
FINAL UPDATED WEIGHT MATRIX FOR TEST CASE 2:
================================================
0.33    0.20    0.10    0.80    0.40
0.51    0.60    0.70    0.90    0.20


students@students-HP-280-G3-SFF-Business-PC:~/Desktop/2024300170_Vedant$
```

```
students@students-HP-280-G3-SFF-Business-PC:~/Desktop/2024300170_Vedant$ g++ lvq.cpp -o lvq
students@students-HP-280-G3-SFF-Business-PC:~/Desktop/2024300170_Vedant$ ./lvq
================================================
       LVQ (Learning Vector Quantization)
================================================
Initial Reference Weights:
w1 (Class 1): [ 0.00 0.00 1.00 1.00 ]
w2 (Class 2): [ 1.00 0.00 0.00 0.00 ]

================================================
                    EPOCH 1
================================================
------------------------------------------------
Training Vector 1: [ 0.00 0.00 0.00 1.00 ] | Target Class (T): 2
------------------------------------------------
D(1) to w1 = 1.00
D(2) to w2 = 2.00

Winning Unit: J = 1 (Class 1) with distance = 1.00
Mismatch! J != T (1 != 2)
Updating w1 using subtraction (-)...
Updated w1: [ 0.00 0.00 1.10 1.00 ]

------------------------------------------------
Training Vector 2: [ 1.00 1.00 0.00 0.00 ] | Target Class (T): 1
------------------------------------------------
D(1) to w1 = 4.21
D(2) to w2 = 1.00

Winning Unit: J = 2 (Class 2) with distance = 1.00
Mismatch! J != T (2 != 1)
Updating w2 using subtraction (-)...
Updated w2: [ 1.00 -0.10 0.00 0.00 ]

------------------------------------------------
Training Vector 3: [ 0.00 1.00 1.00 0.00 ] | Target Class (T): 1
------------------------------------------------
D(1) to w1 = 2.54
D(2) to w2 = 1.70

Winning Unit: J = 2 (Class 2) with distance = 1.70
Match Found! J = T (2 == 2)
Updating w2 using addition (+)...
Updated w2: [ 0.61 -0.80 0.00 0.39 ]

------------------------------------------------
Training Vector 2: [ 1.00 1.00 0.00 0.00 ] | Target Class (T): 1
------------------------------------------------
D(1) to w1 = 3.02
D(2) to w2 = 3.55

Winning Unit: J = 1 (Class 1) with distance = 3.02
Match Found! J = T (1 == 1)
Updating w1 using addition (+)...
Updated w1: [ 0.10 0.67 1.19 0.33 ]

------------------------------------------------
Training Vector 3: [ 0.00 1.00 1.00 0.00 ] | Target Class (T): 1
------------------------------------------------
D(1) to w1 = 0.27
D(2) to w2 = 4.76

Winning Unit: J = 1 (Class 1) with distance = 0.27
Match Found! J = T (1 == 1)
Updating w1 using addition (+)...
Updated w1: [ 0.09 0.70 1.17 0.30 ]

>>> All training vectors correctly classified! Stopping training process. <<<
```

	``` == TRAINING COMPLETED! Final Updated Weights Matrix: w1 (Class 1): [0.09 0.70 1.17 0.30] w2 (Class 2): [0.61 -0.80 0.00 0.39] == Do you want to predict output for a custom test vector? (y/n): y Enter 4 elements for test vector (x1 x2 x3 x4): 0 0 0 1 Calculated Distances: D(1) to w1 = 2.36 D(2) to w2 = 1.38 == PREDICTED OUTPUT CLASS: CLASS 2 == Predict another vector? (y/n): y Enter 4 elements for test vector (x1 x2 x3 x4): 1 1 1 0 Calculated Distances: D(1) to w1 = 1.04 D(2) to w2 = 4.55 == PREDICTED OUTPUT CLASS: CLASS 1 == ```
CONCLUSION:	In conclusion, this experiment successfully demonstrated the fundamental principles and implementation of unsupervised Kohonen Self-Organizing Feature Maps (SOM) and supervised Learning Vector Quantization (LVQ) networks. By computing distance metrics and applying learning rate adjustments, we observed how input vectors are accurately mapped, clustered, and classified. These networks provide robust and efficient frameworks for dimensionality reduction, feature mapping, and pattern recognition tasks.